

MS1-Level Quantitation for DDA in Pioneer

Design-matrix deconvolution over MS1 scans: implementation and test plan

Pioneer.jl — feat/dda-ms1-quant

2026-08-11

1 Why this is needed (measured, not assumed)

Two files of LFQBench PXD028735 (LFQ_Orbitrap_DDA_Condition_{A,B}_Sample_Alpha_01, QE HF-X, 150 min, HYE mix) were converted and searched end-to-end with the unmodified DIA pipeline. Findings that motivate this work:

- **There is no MS2 chromatogram in DDA.** Across identified precursors, `n_scans` has median 1, mean 1.001, max 3. Dynamic exclusion means a precursor is fragmented essentially once.
- **Integration therefore returns exactly zero.** Instrumenting the two filters in `process_final_psms!` gave, per file: `dropped_nan=0`, `dropped_nonpos=86,143/83,229`, `zero=86,143/83,229`, `neg=0`. Never NaN, never negative — the trapezoid over a single-point trace is identically 0.
- **That deleted 96% of identifications.** Removing the `peak_area > 0` filter (commit 50fb3eb6a) took precursor rows from 7,245 to 176,617 and recovered exactly the pre-integration $q \leq 0.01$ set (89,019 / 86,377), at unchanged decoy rate (0.69%).
- **The IDs are alive but unquantified.** 169,372/176,617 rows (95.9%) still carry `peak_area = 0`.

Success metric for this work: the fraction of $q \leq 0.01$ rows with a non-zero abundance goes from $\sim 4\%$ to near 100%, and those abundances reproduce the known HYE ratios.

2 What exists in the tree today

2.1 Dead code that cannot simply be revived

[IntegrateChromatogramsSearch/utils.jl](#) lines **1798–2049** contain the entire historical MS1 `build_chromatograms` inside a `#= ... =#` block. It calls `buildDesignMatrixMS1!`, `MzGroupingMap`, and `distribute_ms1_coefficients!`, which were added in 1fba8b915 (2025-09-23) to `CommonSearchUtils/buildDesignMatrix.jl`. **That file no longer exists and none of those three symbols have a live definition anywhere in `src/`.** The classic `selectRTIndexedTransitions!` / `matchPeaks!` / `buildDesignMatrix!` / `PrecursorMatch` machinery they were written against was replaced by the fused architecture. So this is a *port*, not an uncomment.

2.2 Live machinery we will reuse

Component	Location
MS2 scan loop to mirror (build_chromatograms)	IntegrateChromatogramsSearch/Utils.jl ~1540–1795
RT-window candidate collection	<code>collect_rt_window_precursors!</code> (utils.jl:1397)
Per-scan m/z pre-correction	<code>prepare_scan_peaks!</code>
Deconvolution solvers	utils/SpectralDeconvolution/solveHuber.jl:268 etc.
Solver selection	<code>calibrated_chromatogram_deconvolution_solver</code>
Sulfur-aware isotope splines	<code>IsotopeSplineModel</code> (utils/isotopeSplines.jl:89)
Exact elemental isotopes	<code>getElementalComposition</code> , <code>getIsotopes</code> , <code>QRroots</code> (utils/isotopes.jl:100)
MS1 mass-error model	<code>getMs1MassErrorModel</code> (SearchTypes.jl:556)
Output row type	<code>MS1ChromObject</code> (structs/ChromObject.jl:49)
Downstream integrator	<code>integrate_chrom</code> (integrate_chrom.jl:530)

`MS1ChromObject` already has exactly the shape we need and requires no change:

```
struct MS1ChromObject <: ChromObject
    rt::Float32; intensity::Float32; m0::Bool
    n_iso::UInt8; scan_idx::UInt32; precursor_idx::UInt32
end
```

The MS1 mass-error model already calibrates well on DDA: bias 0.1/0.21 ppm, tolerance ± 9.2 ppm, from 9,336/9,538 residuals per file. We inherit it for free.

3 Design

3.1 Shape of the problem

The MS1 loop is *structurally simpler* than the MS2 one. In MS2, each precursor contributes tens of fragment rows and requires the fused match/score/build kernel. In MS1, each precursor contributes only its isotope envelope — M_0, M_1, M_2 (optionally M_3). There is no fragment lookup, no NCE model, no spectral scoring. The design matrix is ~ 3 non-zeros per column.

For MS1 scan s with observed peaks $y \in \mathbb{R}^m$:

$$\min_{w \geq 0} \rho(Hw - y), \quad H_{ki} = \tau_i \cdot \alpha_{i, \iota(k)}$$

where column i is a candidate precursor, row k an observed peak, $\alpha_{i,j}$ the predicted relative abundance of isotope j for precursor i , $\iota(k)$ the isotope index matching peak k , and ρ the existing Huber loss. $\tau_i \equiv 1$: **there is no quadrupole transmission term at MS1**, which is the main structural difference from the MS2 path.

3.2 Per-scan algorithm

1. Iterate scans with `getMsOrder(spectra, i) == 1`. (File A has 26,352 MS1 scans vs 109,507 MS2; the MS1 loop is $\sim 4\times$ cheaper in scan count.)
2. **Candidate selection.** Reuse the RT window logic: `rt_tol_local = irt_tol / max(local_slope, 0.01)`, then walk `rt_index` bins. *This requires an MS1 variant of `collect_rt_window_precursors!`*: its current signature ([utils.jl:1397](#)) takes `quad_transmission_func`, `precursor_transmission`, and `min_fraction_transmitted`, all of which are meaningless without an isolation window. Simplest approach: add a method that omits the quad filter rather than passing a `NoQuadModel` sentinel through the existing one.

3. **Peak pre-correction** via `prepare_scan_peaks!` using the **MS1** mass-error model (`getMs1MassErrorModel`), not the MS2 one. The historical code got this wrong once already — see 9b46e745a, “use MS1 mass error model in `ScoreFragmentMatches!` (replace MS2 model)”.
4. **Build H** . For each candidate i and isotope $j \in \{0, 1, 2\}$, compute $m/z_{ij} = (\text{mono_mass}_i + j \cdot \Delta_{13\text{C}})/z_i + \text{proton}$, binary-search the peak list within the MS1 ppm tolerance, and if matched write α_{ij} into the sparse column. Reuse the cursor-based lookup primitives already written for MS1 features (`_ms1_find_peak_from_cursor` and friends, [MainSearch/features.jl:295--360](#)) rather than writing new search code.
5. **Solve** with `solve_deconvolution!` and the configured solver.
6. **Emit** one `MS1ChromObject` per candidate per scan (zero weight when unmatched), exactly as the MS2 path emits `MS2ChromObject`.

3.3 Collinearity: grouping plus ridge

At MS1 there is no fragmentation, so **two precursors with the same m/z and charge are perfectly collinear** and their individual weights are unidentifiable. This is not an edge case: Pioneer’s decoys are sequence permutations, so a decoy has the *same elemental composition* — hence the same precursor m/z , the same charge, and the same isotope envelope — as its target. **Every target/decoy pair is an exactly duplicated column.** Isobaric distinct peptides (the VISSIEQK/IVSSIEQK pair is a known case in this codebase) add more.

Resolution: collapse exact duplicates, then let ridge handle the rest

Two mechanisms, covering two different failure modes. Neither substitutes for the other.

Exact grouping → **exactly-collinear columns**. Before building H , group candidates by the key (**rounded m/z , charge**). Emit *one representative column per group* into the solve. After fitting, assign the group’s weight back to every member. No amount of regularisation can separate identical columns, so these must be collapsed structurally.

Light ridge → **near-collinear columns**. Distinct peptides within the MS1 tolerance (± 9.2 ppm as fitted on this data) match the *same* observed peaks, so their columns share support and differ only slightly in their α ratios. These are ill-conditioned but not identical; an ℓ_2 penalty is the right tool.

The grouping key must include charge. The historical `MzGroupingMap` keyed on m/z alone, rounded to 10^{-5} . That is incorrect: two precursors with equal m/z but different charge have isotope spacing $\Delta = 1.00336/z$, so their M_1 and M_2 rows fall at *different* m/z . Their columns are not identical and merging them would introduce a real error. Keying on (m/z , charge) fixes this.

Rounding precision. 10^{-5} is ~ 0.02 ppm at m/z 500, far finer than the ± 9.2 ppm MS1 tolerance, so it collapses *only* exact duplicates — which is precisely the intended division of labour above. Keep it.

The group weight is duplicated, not split

After the solve, each member of a group receives the **full** group coefficient. This is what the historical `distribute_ms1_coefficients!` did:

```
# Assign this coefficient to all precursors in the group
for prec_id in mz_grouping.mz_group_to_precids[mz_group]
    precursor_weights[prec_id] = group_coefficient
end
```

Splitting the weight n ways is the intuitive alternative and it is *wrong*: it would systematically underestimate every real peptide in proportion to how many isobars happen to share its m/z — an abundance bias driven by library composition rather than by chemistry. Duplication gives each candidate the correct conditional quantity, “the abundance *if* this is the peptide,” and the MS2 identification decides which member that is. Only the identified member survives FDR, so the duplicated values on the others are never consumed.

Consequence to keep in mind: a target and its decoy receive identical MS1 weights by construction. That is safe for FDR (the value cannot discriminate, so it introduces no artificial separation), but it does mean **MS1 quant can never contribute a discriminating feature to scoring**. It is a quantitation mechanism only.

3.4 Solver and regularisation

Decision: Huber with a light λ for the initial implementation. It is the only solver that already supports a ridge penalty — `HuberSolver` carries `lambda` and `reg_type`, and `L2Norm` is implemented (`getRegL2` returns 2λ , [solveHuber.jl:36](#)).

Note the MS2 chromatogram path currently runs *unregularised*:

```
function default_chromatogram_integration_huber_solver()
    return HuberSolver(300.0f0, # delta
        0.0f0, # lambda (no regularization)
        ..., NoNorm())
```

so the MS1 path needs its own solver instance with $\lambda > 0$ and `L2Norm()` rather than reusing the MS2 default.

Gap to close later: `OLSSolver` and `PoissonMMSolver` are empty singletons (`struct OLSSolver <: DeconvolutionSolver end`) with no penalty term at all, so “ridge + OLS” and “ridge + PMM” do not currently exist. Adding an ℓ_2 term to both, following the pattern already in [solveHuber.jl](#), is a small but real piece of work. Sequencing: **Huber+ridge first, get MS1 quant working end-to-end, then add ridge to OLS/PMM and compare all three** (test T7). Carry λ as a parameter on the MS1 path from the start so no plumbing changes when the other two solvers gain it.

3.5 Isotope abundances: splines vs. exact composition

Both are live in the tree.

Sulfur-aware averagine splines. `iso_splines(S, I, mono_mass)` ([isotopeSplines.jl:89](#)), Goldfarb et al. 2018. Indexed by `sulfur_count` and isotope index. Note this is *not* plain averagine: sulfur is the dominant deviation from averagine at M_2 (via ^{34}S at 4.25%), and it is already corrected for. `sulfur_count` is carried per precursor in the library (`getSulfurCount, precursors.jl:81`). $O(1)$ spline evaluation; already used in the MS2 hot path and by `collect_rt_window_precursor`.

Exact elemental composition. `getElementalComposition(seq) + getIsotopes(comp, roots, npeaks, charge, id)` using `QRroots` ([utils/isotopes.jl](#)). Exact, but: (i) it needs the formula per precursor, and (ii) `mod.to_composition` **currently contains exactly one entry**, “ox” => `Composition(0,-1,0,1,0)`. Any other modification would be silently mis-composed. Extending that table is required before this path is trustworthy.

Recommendation: start with the sulfur-aware splines, for three reasons: they are already validated in this codebase, they cost nothing extra per scan, and the $M_0:M_1:M_2$ ratios they produce are accurate to well under the noise of a single MS1 scan for tryptic peptides once sulfur is accounted for. **But make the abundance source a function argument, not a hard-coded call**, so the

exact path can be swapped in and A/B'd on the same data without touching the loop. Test T3 below is designed to measure whether it matters.

4 Where it plugs into the pipeline

MS1 deconvolution runs inside `IntegrateChromatogramsSearch`, alongside the existing MS2 extraction, producing a second chromatogram table. Then:

1. Both tables feed `integrate_chrom. avg_cycle_time` is already computed *per trace* as `(rt_last - rt_first)/N` ([utils.jl:209](#)), so irregular DDA sampling needs no change — and MS1 sampling is far more regular than MS2.
2. `peak_area` for a DDA run is taken from the MS1 trace. The MS2 trace is retained for its existing role in scoring/MBR.
3. The `peak_area > 0` filter, currently disabled outright, becomes the intended DDA/DIA switch: with MS1 quant supplying real areas, a zero area again means something.

Integration width. A separate but adjacent question. The candidate-inclusion rule is scan-centric and symmetric — precursor i enters scan s 's solve iff $|RT(s) - RT_{\text{pred}}(i)| \leq \text{rt_tol_local}$, anchored on *library-predicted* iRT, not on the observed best scan. `irt_tol` measures iRT prediction error and is acquisition-independent, so it carries over from DIA unchanged. At MS2 in DDA the 1.3 Th isolation window prunes candidates hard; **at MS1 there is no such pruning, so `rt_tol` alone bounds the design-matrix width.** Expect materially wider matrices at MS1 and budget for it (§6).

5 Test and validation plan

Staged so each step is falsifiable before the next is built.

- T0 — Golden regression (must run first).** The MS1 path must be inert for DIA. Run the existing Olsen Exploris and MTAC 3P configs before and after; precursor counts, q-values, and `peak_area` must be *bit-identical*. Gate the whole MS1 path behind a flag so this is guaranteed by construction, and verify rather than assume.
- T1 — Single-precursor sanity.** Pick ~ 20 high-confidence HUMAN precursors from `dda_run4_nofilter`. For each, dump the MS1 XIC and check by eye: (i) is there a peak, (ii) does its apex sit at the MS2 detection RT, (iii) is the width physically plausible ($\sim 10\text{--}30$ s for a 150 min gradient). *This is the “do the chromatograms make sense” step and it gates everything after it.*
- T2 — Envelope consistency.** For matched precursors, compare observed M_1/M_0 and M_2/M_0 against predicted. Systematic deviation indicates a wrong abundance model or a mass-tolerance problem; scatter indicates interference. Reuses `ms1_m1_to_m0_ratio / ms1_m1_to_m0_pred`, which the feature code already computes.
- T3 — Splines vs. exact composition.** With the abundance source parameterised, run both on the same file and compare T2 residuals and final CV. **Pre-registered decision rule:** adopt exact composition only if it measurably improves envelope residuals *and* the `mod_to_composition` table has been extended to cover every mod in the HYE library. Otherwise keep splines.
- T4 — Coverage.** Fraction of $q \leq 0.01$ rows with `peak_area > 0`. Baseline today: 4.1%. Target: $> 90\%$.

- T5 — Accuracy and precision on the full A/B set.** Convert the remaining 22 Condition A/B files, then evaluate: (i) log-ratio distributions per species against the known HYE design — HUMAN centred at 0, YEAST and ECOLI at their design ratios; (ii) CV within the 4 replicates of each sample group (Alpha/Beta/Gamma), which gives a precision estimate independent of the fold-change check.
- T6 — External comparator.** Sage on the same files (already at `~/Projects/Sage`) for ID count and quant accuracy. Compare with MBR off on both sides for an apples-to-apples result.
- T7 — Solver comparison (deferred).** After Huber+ridge is working end-to-end and an ℓ_2 term has been added to `OLSSolver` and `PoissonMMSolver`, compare all three on identical input at matched λ . Metrics: envelope residual (T2), coverage (T4), replicate CV and fold-change accuracy (T5). Poisson is a priori attractive here because MS1 peak intensities are closer to counting statistics than the heavy-tailed interference Huber was chosen to resist, but this is a hypothesis to test, not an assumption to build on.
- T8 — λ sensitivity.** Sweep the ridge penalty over ~ 3 – 4 decades on one file. Expect a plateau; pick from the middle of it rather than the optimum, and record the curve so the choice is auditable. A sharp optimum would itself be a warning that the problem is more ill-conditioned than §3.3 assumes.

5.1 Disk budget for T5

22 further raws at ~ 3.7 GB = ~ 81 GB; only ~ 77 GB is free. Arrow output is ~ 430 MB/file ($\sim 12\times$ smaller), so **convert-and-delete each raw in turn**; the full converted set is only ~ 10 GB. Alternatively reclaim from the 56.8 GB of `RegressionTestsLite` run outputs.

6 Risks and things that could invalidate this

1. **Design-matrix width at MS1.** With no isolation-window pruning, every library precursor within `rt_tol` of the scan is a candidate. Against a 5.25M-precursor library this could be far larger than the MS2 case. Mitigation: restrict columns to the *passing* precursor set (the same `precursors_passing` the MS2 path already uses) — roughly 10^5 rather than 10^7 . Must be measured early, not assumed.
2. **Group weights are not per-precursor abundances.** Stated in §3.3. If apportionment cannot be made principled, MS1 quant is really “abundance of this m/z group”, which is honest but must be labelled as such.
3. **MS1 interference in a complex background.** HYE at 150 min is dense; co-eluting isobars are common. T2 measures this directly.
4. **Splines wrong for modified peptides.** Splines are indexed by mass and sulfur only. A heavily modified peptide’s envelope may deviate. T3 is the check.
5. **DIA regression.** Any change to shared code (e.g. `collect_rt_window_precursors!`) risks the DIA path. T0 guards this; prefer adding methods over editing existing ones.

7 Open questions for review

Resolved since the first draft: group-weight apportionment (§3.3 — collapse exact (m/z, charge) duplicates to one column, duplicate the weight back to all members, never split) and

solver choice (§3.4 — Huber + light ridge now, OLS/PMM ridge later). Charge is no longer a separate question: it is part of the grouping key.

1. **Number of isotopes.** $M_0..M_2$, or include M_3 for heavier precursors? More isotopes give more rows per column (better conditioning) but more interference. Suggest starting at $M_0..M_2$ and testing M_3 as a variant under T2.
2. **Should decoys be deconvolved at all?** Given §3.3, every decoy is an exact duplicate of its target and gets collapsed into the same column anyway — so including them costs almost nothing after grouping. The question is now mainly whether carrying them buys anything. Leaning: keep them, since grouping has made them cheap.
3. **Does MS2 quant stay?** Proposal is to keep the MS2 trace for scoring/MBR and take `peak_area` from MS1 in DDA mode. Confirm.
4. **Where does the DDA/DIA switch live?** A config flag, or inferred from the data (e.g. variable `centerMz` across MS2 scans, which is already how DDA is detectable — see the `cycle_idx` collapse)? Inference is more convenient but a silent mode switch on a heuristic is a debugging hazard.

8 Proposed milestones

1. Flag-gated skeleton + MS1 variant of `collect_rt.window_precursors!`; T0 green.
2. (m/z, charge) grouping + representative-column selection; assert one column per distinct key and that every target/decoy pair collapses.
3. Design-matrix build with spline abundances; log matrix dimensions per scan (risk 1).
4. Solve with Huber + L2Norm light λ ; emit `MS1ChromObject`; distribute weights back to group members. T1 chromatogram inspection. **Review gate.**
5. Wire into `integrate_chrom`; T2, T4, T8.
6. T3 spline-vs-exact A/B.
7. Convert remaining 22 files; T5, T6.
8. *Deferred*: add ℓ_2 to `OLSSolver` / `PoissonMMSolver`; T7.